

Week 1: learn, build and show evidence

Public learning edition / Rizwan Alam / 40 hours

Your only technical outcome this week: run a minimal simulated 5G standalone lab, demonstrate registration plus a PDU session plus user traffic, and preserve enough evidence to explain and repeat the result.

This is a standalone expansion of Week 1 in your personalized roadmap. It uses your existing IMS, C/Rust and Linux background. It is a work plan, not a claim that the lab or its tests have already been executed.

The Friday demonstration

In five minutes, show the topology, start the lab using documented commands, demonstrate UE registration and session establishment, send traffic through the UE data path, and point to the matching capture/log evidence. Explain one source-code path and one visibility limitation. Repeat the happy path after a controlled stop/start if time and stability allow.

What you will have produced

- A one-page scope, environment manifest and version ledger.
- A small topology diagram and configuration-consistency checklist.
- Build/test logs for the selected core and simulator versions.
- A baseline run folder containing captures, selected logs and traffic results.
- An annotated procedure timeline, manual runbook and Friday assessment.

Focus rules

Use one Linux lab environment and one simulated UE. Keep the selected versions fixed during the baseline attempt. Reuse existing documentation or code only if it directly reduces work. This week does not require a new Rust parser, an AI agent, a Kubernetes deployment, a new upstream PR or a public post.

Your existing Kamailio and rsipstack relationships remain valuable; cap urgent review follow-up at 30 minutes within the daily buffer. Queue new contribution ideas for later.

Navigation

Page 2: schedule. Pages 3-7: daily execution. Pages 8-12: evidence, troubleshooting and checks. Pages 13-17: timed task checklists. Pages 18-20: public deliverables, progress updates and sharing.

[Public learning repository](#). The live dashboard is the current record; this PDF is the plan.

Your 40-hour schedule

Five days, eight work hours per day

Use these as elapsed work blocks, not fixed clock times: **Block A = 2 hours; B = 2 hours; C = 2 hours; D = 1 hour; E = 1 hour.** Lunch and short recovery breaks are outside the 40 work hours. Start/finish times can vary; keep the dependency sequence.

Day 1: A scope and existing-work inventory; B skill demonstration; C Linux environment decision; D workspace and evidence structure; E review/buffer.

Day 2: A host/guest preflight; B transport/TUN and topology checks; C dependency/version selection; D environment manifest and snapshot; E review/buffer.

Day 3: A Open5GS dependency setup/build; B core tests and build diagnosis; C UERANSIM build; D configuration consistency; E review/buffer.

Day 4: A core startup/health; B gNB connection; C UE registration and PDU session; D user traffic; E review/buffer.

Day 5: A clean captured run; B annotation and source walk; C controlled repeat; D demonstration and assessment; E packaging/buffer.

Total and contingency

Each day has 7 planned task hours plus 1 buffer hour. Total = 35 task hours + 5 buffer hours = 40. The buffers absorb setup friction, concise notes, urgent review follow-up and unfinished prerequisite work. They are not extra features.

The annual roadmap's category allocations are averages. This first week intentionally concentrates on setup and baseline evidence. Do not add the annual reading, research and contribution quotas on top of these 40 hours.

Daily closing ritual

Write: what worked; exact command/commit; what failed; evidence path; next single action. Commit only appropriate small configuration/templates/notes to the lab record. Keep large captures, database contents and credentials outside ordinary Git history.

When a day slips

Spend its buffer first. Then move the dependent task to the next day and remove optional work. Do not build on a failing prerequisite to preserve the calendar. If source builds remain blocked after Day 3, Friday's honest deliverable becomes a minimal build/environment blocker package rather than a claimed successful lab.

Day 1: decide what you are proving

Block A / 2 hours / scope and reuse

First 30 minutes: write this target in scope.md: "Prove and explain one simulated UE registration, PDU session and user-data path on a pinned 5GC lab." Add the seven Friday outputs from page 1 and mark each not started.

Next 60 minutes: inspect your existing ims-5gc-iwf deployment notes and local lab material. Record reusable topology, logging or setup knowledge. Treat unexecuted instructions as candidates, not validated dependencies. Spend no time adding interworking features this week.

Final 30 minutes: create a backlog of at most five entries: environment; builds; configuration; happy-path run; evidence/repeat. Put XFRM benchmarking, SIP tool enhancements and AI integration in a separate later list.

Block B / 2 hours / demonstrate your starting knowledge

Spend 30 minutes each on four checks. Draw the 5GC registration/session path from memory, marking uncertainty. Explain one C ownership/state-transition decision from familiar code. Explain one Rust error-handling decision from your own project. Trace a Linux packet through interface, route and firewall decisions.

For each, score 0 = cannot show; 1 = tutorial dependent; 2 = independent explanation; 3 = can diagnose a new failure and defend a tested fix. Attach one note or code/trace reference. This is gap finding, not a beginner exam.

Block C / 2 hours / choose a workable environment

Inventory the existing Windows workstation and available Linux host/VM. Record CPU, RAM, disk, virtualization and network access. Prefer an existing suitable Linux environment; provision a dedicated VM if needed. Do not replace an existing working lab without evidence it is unsuitable.

Choose one topology: one Linux VM with documented addresses for core and simulator is a reasonable starting design. Check address ranges for collisions with host, VPN and existing labs. Hardware estimates in the annual plan are planning estimates, not a reason to purchase equipment today.

Block D / 1 hour / organize evidence

Create the proposed lab workspace on page 10, outside existing project source trees. Add a run-log template and a version ledger. Keep this lab record local until its contents are reviewed for sharing.

Block E / 1 hour / gate

Done: scope, five-item backlog, four starting checks and a chosen Linux path exist. **If blocked:** write exactly which resource or capability is missing. Tomorrow's first task addresses that dependency; do not start a different project.

Day 2: establish the environment

Block A / 2 hours / record the actual machine

Boot the Linux guest/host. Record OS release, kernel, architecture, CPU, available RAM/disk, interface addresses and routes using the read-only checks on page 12. Record VM networking mode and how you reach its console. Save a snapshot before making lab-specific networking changes if the platform supports it.

Install prerequisites using the official instructions that match the chosen OS [W1-W2]. Log commands and outcomes. Resolve a package failure at its first meaningful error; do not keep adding unrelated packages based on the final line alone.

Block B / 2 hours / check capabilities and topology

Check that the TUN device path is available and the guest can perform the lab's required networking operations. Inspect SCTP availability; a missing loaded-module entry alone does not prove it is unsupported because support may be built in. The decisive transport test is the simulator/core connection later.

Draw these logical paths: simulated UE to gNB; gNB to AMF; SMF to UPF; gNB to UPF; UPF to a controlled data endpoint. Annotate which interfaces/addresses carry each path. Internal services may share the same host; a loopback address refers only to its own network namespace.

Block C / 2 hours / select and record versions

Choose explicit releases or commits for Open5GS and UERANSIM, following their official project guidance [W1-W2]. Record full commit IDs, compiler/build-tool versions and documentation links. Do not assume "latest + latest" is a proven compatible pair.

Create a compatibility row for each component: selected version, build result, configuration reference, runtime result and limitation. Runtime result remains not tested today. Keep the compiler/toolchain used by existing projects separate if their constraints differ.

Block D / 1 hour / save a reproducible state

Write environment.md and topology.md. Save package/tool version output. Identify where the database, configuration, binaries and logs will live. Note what must be recreated after reboot versus what is persistent. Make a ready-for-build snapshot if practical.

Block E / 1 hour / gate

Done: Linux is reachable, resources are recorded, required capability checks have no unresolved blocker, and both source revisions are selected. **If blocked:** narrow the failure to VM networking, kernel capability or package/toolchain setup. Do not debug application registration before these foundations exist.

Day 3: build and validate the components

Block A / 2 hours / Open5GS build

Work from the recorded source revision. Follow the OS-specific source-build guide [W1], adapting paths to your lab. Capture dependency installation and compilation output in separate logs. Record the build directory, configuration directory and binary paths.

When compilation fails, preserve the first compiler/configuration error, source revision and relevant dependency version. Reproduce from the same inputs. Avoid changing source, compiler and dependencies simultaneously; you would lose the explanation of what fixed it.

Block B / 2 hours / core tests

Run the documented relevant test entry points for that revision. Save the command, exit status, test summary and failure output. Record any prerequisite such as database or network setup. A compiled binary is not proof that its tests pass.

Label failures precisely: application assertion; missing runtime dependency; permission/capability; external environment. If a test is skipped or cannot run, list it as such rather than treating it as a pass. Do not patch core behavior simply to make the initial environment work.

Block C / 2 hours / UERANSIM build

Build the pinned simulator following its installation instructions [W2]. Confirm expected binaries exist and their help/usage output works. Save the source revision and build log. If the project does not provide an applicable local test suite, record that fact; runtime validation on Day 4 remains required.

Block D / 1 hour / prepare configuration

Copy the relevant example configurations into your lab configuration area with provenance. Compare, rather than blindly copy, PLMN, tracking area, slice selection, DNN, addresses and synthetic subscriber settings [W3]. Keep a private record of test credential agreement without printing keys into a shared report.

Record the database/service prerequisites and startup sequence for the selected core configuration. Distinguish the core's virtual interface from the simulated UE's virtual interface; they serve different roles in the packet path.

Block E / 1 hour / gate

Done: both projects build, relevant available tests have recorded outcomes, and configuration files are identified. **If blocked:** create a minimal reproduction including exact command and first error. Use remaining time to resolve that build; registration waits. A binary from an unrelated prebuilt version must not silently replace your pinned source build.

Day 4: connect one UE end to end

Block A / 2 hours / start and inspect the core

Start the database and required network functions using the selected version's instructions and configuration. Record each process, config path and log path. Check expected listeners and service discovery where applicable. Confirm the UPF-facing network setup and data endpoint before bringing up the UE.

Pass: required services stay running and expose the expected endpoints. A running process by itself is not proof of readiness. Resolve bind failures, missing dependencies or discovery errors first.

Block B / 2 hours / gNB to AMF

Start the simulated gNB with the lab configuration [W3-W4]. Confirm it can reach the configured AMF transport address. Observe successful NG setup in both available logs and capture. Check PLMN/tracking-area consistency if setup is rejected.

Pass: the gNB/core association is established with an evidence pointer. If not, check transport reachability, listening address, namespace boundaries and SCTP support before editing UE credentials.

Block C / 2 hours / registration and session

Provision a synthetic subscriber using the chosen core's supported method. Match its identity and authentication settings to the simulated UE. Start one UE [W3-W4]. Record registration outcome, then requested/established PDU-session outcome separately.

If registration works but the session fails, investigate DNN/slice/subscription and session handling. If the session succeeds, record the assigned UE address and the relevant interface. Do not infer usable data connectivity from address assignment alone.

Block D / 1 hour / controlled user traffic

Choose a known data endpoint that will force traffic through the simulated UE interface and UPF path. Bind the traffic source to that UE path as supported by the environment. Observe packets at the UE side and core/user-plane side; a host's ordinary successful ping is insufficient.

Confirm return traffic as well as outbound traffic. Begin with reachability to a controlled IP endpoint so DNS is not another unknown. Record the traffic command, source interface/address, destination, start/end times and results. See page 8 for evidence expectations.

Block E / 1 hour / gate

Done: registration, session establishment and bidirectional user traffic each have their own evidence. **If blocked:** stop at the first failing layer and follow page 9. Save successful earlier stages; do not mark the whole lab working when only registration succeeds.

Day 5: capture, explain and repeat

Block A / 2 hours / create the golden run

Start from a documented stopped state. Create a new run ID. Record versions, config hashes and start time; start captures before the procedure. Start core/gNB/UE using the runbook, run the same controlled traffic and stop recording cleanly. Keep only this run's evidence together.

Verify captures contain the intended interfaces and time window, logs are readable, and traffic actually traversed the intended path. A file that exists but contains no useful packets does not satisfy the evidence requirement.

Block B / 2 hours / explain the observed path

Spend 75 minutes constructing the timeline on page 8. Mark each row as observed, inferred or unavailable. Spend 45 minutes locating one related handler/state transition in the pinned source. Record file/function/commit and explain how the observed event connects to that code.

Choose one manageable question: how is the observed registration/session event handled, or where is its timeout/error reported? Do not try to read all network functions. Source proximity alone does not prove runtime execution; distinguish code interpretation from captured runtime evidence.

Block C / 2 hours / controlled repeat

Stop the lab cleanly and repeat startup, UE/session creation and the same traffic probe. Save a second run ID. Compare procedure outcomes, addressing and packet counts qualitatively; timestamps and allocated identifiers can legitimately differ.

If it fails, record the earliest divergence: persistent session state, stale interface, listener collision, service discovery or another observed cause. Fix only what the evidence supports. This repeat is a Week 1 confidence check; complete reset automation and broader repetition belong later.

Block D / 1 hour / demonstrate and assess

Deliver the five-minute demonstration to a peer if available, otherwise record it locally. Spend the rest of the hour scoring page 11 and writing the remaining gaps. State exactly what the simulation proves and what it does not.

Block E / 1 hour / package

Finish the README/runbook, confirm evidence references open, and save your notes/configuration safely. Select the first next action only; do not start Week 2 during this block. **Done:** another engineer can understand the run and follow the documented reproduction without relying on your memory.

Capture and annotation checklist

Capture with a question in mind

Before each run, list the observation points needed for registration, session setup and user traffic. Include loopback or internal interfaces where relevant to this topology. Avoid capturing unrelated host/VPN traffic. Record capture commands, interfaces, tool versions and any dropped-packet statistics.

Save original captures unchanged. Put annotations in a separate file with capture filename, packet number/time and interpretation. A decoded view can omit encrypted content or unsupported messages; record that gap. Do not claim every NAS/SBI field is observable in every setup.

Build an eight-row evidence timeline

1. Core/gNB readiness: where you observed successful setup and which log/capture supports it.
2. Registration initiation: UE/core evidence and visible protocol metadata.
3. Authentication/security progression: only what your evidence actually exposes.
4. Registration outcome: positive result or exact failure evidence.
5. Session request: requested DNN/slice/session information where visible.
6. Session establishment: control-plane outcome and assigned UE address.
7. User data: proof that the intended flow traversed the simulated UE path and user plane.
8. Return traffic and stop: response observed and how processes/captures were closed.

For every row record timestamp, source, evidence pointer, observed fact, interpretation and uncertainty. A blank or "not observable" field is preferable to an invented event.

Match identities cautiously

Record whichever session identifiers are available, but do not assume one identifier is universal across protocols. A TEID needs endpoint/direction/lifetime context. A timestamp match suggests correlation; it does not establish causation. Use one UE this week to reduce ambiguity without pretending the method already scales.

Probe selection

Use a controlled endpoint and interface-bound traffic. Ensure the packet route cannot bypass the UE tunnel through the host's normal interface. If two components share a host, inspect the actual route and observation points explicitly. Show a matching request/response pair and the source/destination used.

Expected artifact: timeline.md plus raw capture/log references, with a short paragraph explaining supported visibility and missing evidence. Packet counts and latency this week are sanity checks, not publication-quality performance results.

Troubleshoot from the first failing layer

Environment or build fails

Preserve OS/kernel/tool versions and the first error. Check the documented prerequisites for the selected revision. Make one change, rerun the failing step and record its effect. If blocked for 90 minutes without a narrower hypothesis, write a five-line blocker note and simplify the setup.

gNB does not connect

Check that the AMF listens at the address the gNB actually uses. Confirm reachability across the chosen namespaces/VM network, permitted SCTP traffic and relevant transport support. Check for port/address collisions. If the transport connects but setup is rejected, examine the reported configuration/protocol cause.

UE registration fails

Start with the exact observed failure stage. Compare synthetic subscriber provisioning and UE authentication/identity settings privately. Check selected PLMN and service reachability. Avoid repeatedly changing subscriber, slice and routing settings together. Save the before/after configuration difference without exposing keys.

Registration succeeds, session fails

Check requested DNN/slice against the subscriber and selected core configuration. Inspect SMF/UPF reachability and related session-control logs. Establish whether the failure is selection/subscription, transport, session handling or an unavailable dependency from actual evidence.

Session succeeds, traffic fails

Confirm the assigned UE address and source interface used by the probe. Inspect the UE-side route, tunnel endpoints, UPF data-network path and return route. Observe where packets stop. Check firewall/forwarding/NAT only as required by the chosen design; do not globally disable protections or flush existing rules to make a test pass.

A retry works but a clean run does not

Compare process state, database/session state, stale interfaces, routes and old listeners. Save the failed run alongside the successful one. A warm-state workaround is a limitation until reproduced and explained.

Fallback that still produces useful evidence

If the complete lab is blocked, deliver the pinned environment, build/test record, last successful layer, minimal failing command, relevant logs and next hypothesis. This is a blocked Week 1 outcome, not completion. Shift dependent work; preserve the quality standard rather than claiming success from partial screenshots.

Copy these planning templates

Proposed workspace

Use one lab workspace containing **docs/** for scope/topology/runbook; **config/** for reviewed lab templates; **scripts/** for documented helpers; and **runs/** for local run evidence. These are proposed relative folders, not files already created by this plan. Keep source checkouts separate and link their exact locations in `environment.md`.

Each run folder should have **manifest**, **capture/**, **logs/**, **traffic/** and **notes.md**. A simple ID such as `w1-d5-run01` is sufficient. Large captures and secret-bearing files should not enter the normal repository accidentally.

scope.md

Goal: one simulated UE, one lab topology, one successful registration/session/data flow.

Success evidence: named run folder, timeline and repeat result.

Constraints: 40 hours; known Linux environment; no requirement for AI or new parser work.

Excluded work: independent IMS integration, XFRM optimization, Kubernetes migration, PR quota.

Open questions: host capabilities; exact source versions; transport and return routing.

environment.md

Record date/timezone; host/guest OS; kernel; architecture; resources; VM networking; interface/subnet map; TUN/transport checks; component full commit IDs; compiler/build tools; binary/config/log paths; dependency versions; setup documentation URL and date; snapshot identifier; limitations.

run manifest

Record run ID; start/end UTC time; component versions; configuration hashes; subscriber alias without credentials; topology version; capture points; traffic command; readiness outcome; registration outcome; session outcome; traffic outcome; stop/reset steps; errors; evidence file paths.

Five-line blocker note

Expected behavior. Actual behavior. Smallest command/configuration that reproduces it. Evidence and last known successful layer. Next discriminating check. Add a change history if more than one attempted fix exists.

Daily log

Hours used by block; artifact completed; verification performed; evidence path; important decision; unresolved dependency; tomorrow's first action. A useful entry says "PDU session succeeds, no return packets at UPF data interface" rather than "worked on networking all day."

Friday: judge completion honestly

Required checks

- **Environment:** another reader can identify the actual kernel, revisions, build tools, topology and configuration locations.
- **Build:** both required components have build results; relevant test outcomes are recorded with failures/skips distinguished.
- **Registration:** the UE's successful registration has a precise evidence pointer.
- **Session:** session establishment and assigned address are independently evidenced.
- **Data:** request and response traverse the intended UE/user-plane path; the result is not the host's ordinary network connectivity.
- **Explanation:** the timeline separates fact, inference and unavailable information; one source-code path is described at a pinned revision.
- **Reproduction:** commands, initial state and a controlled repeat outcome are documented.

Outcome labels

Complete: the required happy path and evidence exist; a controlled repeat succeeds with explained differences. **Provisional baseline:** one successful run exists but repeatability or documentation is incomplete. **Blocked:** a prerequisite or flow stage prevents the demonstration. These labels guide next work; they are not a score of your professional ability.

Five-minute demonstration script

Minute 1: show the topology and what you are testing. Minute 2: show core/gNB/UE readiness and registration/session result. Minute 3: run or replay the recorded traffic result with matching packet evidence. Minute 4: explain one timeline/source connection. Minute 5: explain limitations and the next unresolved question.

If showing recorded output, say so; do not describe it as a live run. If a peer cannot reproduce it, record the exact gap and improve the runbook.

The only handoff to next week

Write one sentence naming the earliest unfinished dependency. If all required checks pass, name the first small step toward automated start/health/reset or the first controlled fault. Stop there. The purpose of this guide is to finish Week 1, not to restart the whole annual plan.

Personal progress question

Can you now explain how a 5GC flow differs from the familiar IMS/Linux behavior in your existing work, with concrete evidence? That connection is more valuable this week than another repository, course certificate or broad list of technologies.

Read-only checks and implementation sources

Example commands inside the Linux guest

Run these individually in a Linux shell; they are not PowerShell commands. Save relevant output locally and review addresses/usernames before sharing it. They inspect state and do not configure the lab.

OS/kernel: `uname -a ; cat /etc/os-release`

Resources: `lscpu ; free -h ; df -h`

Network: `ip -brief address ; ip route show`

TUN path: `ls -l /dev/net/tun`

Versions: `git --version ; gcc --version ; cmake --version ; meson --version`

In each source checkout: `git rev-parse HEAD ; git status --short`

The semicolons above separate independent examples. A missing command is an environment observation; install a tool only when required. Device presence is not proof that all needed operations work. Record actual capability and runtime checks separately.

Current upstream instructions

[W1] [Open5GS source build and tests](#). Use this for version/OS-appropriate dependency, build and core configuration guidance. Read the relevant section before executing privileged setup commands.

[W2] [UERANSIM installation](#). Use for simulator platform/build prerequisites and installation procedure.

[W3] [UERANSIM configuration](#). Use for the actual selected configuration fields; reconcile values with your core and subscriber setup.

[W4] [UERANSIM usage](#). Use for the selected version's gNB/UE startup and runtime operations.

Scope of verification

These pages were checked during preparation. The commands and lab procedures have not been executed against your Linux environment. Exact install/configuration commands must match the OS, topology and commits you choose; this document deliberately does not supply a universal installation script.

This Week 1 guide builds on the preceding profile review and your confirmed 40-hour commitment. It does not re-audit contribution statuses or change your GitHub, LinkedIn or existing repositories.

Day 1 task checklist: Scope, starting point and topology

Budget: 7 task hours + 1 hour buffer. All checkboxes start unchecked. Complete them only when the work is actually performed.

A - Scope and reuse (2h)

- 0-30m: fill D01 with the single-UE goal, explicit non-goals and Friday evidence.
- 30-60m: inventory existing deployment notes and tooling; put links and exact useful parts in D02.
- 60-90m: decide what can be reused without adding dependencies. Do not extend existing project features.
- 90-120m: limit the active backlog to environment, builds, config, baseline and evidence. Mark priorities in D01.

B - Starting knowledge (2h)

- 0-30m: draw registration/session flow from memory; label uncertain steps.
- 30-60m: explain ownership and a state transition in familiar C code. Record its file and revision.
- 60-90m: explain a Rust error-handling decision and one failure case from your work.
- 90-120m: trace a Linux packet through interface, routing and firewall decisions. Score each check 0-3 with evidence in D02.

C - Environment and topology (2h)

- 0-30m: inventory available Linux host/VM options and access.
- 30-60m: select one environment; record why it fits and what remains untested.
- 60-90m: draw UE, gNB, AMF, SMF, UPF and a controlled data endpoint.
- 90-120m: annotate N2/N3/N4 and the data path with proposed addresses/namespaces; check host/VPN subnet collisions in D03.

D - Organize the learning record (1h)

- 0-20m: create private raw-evidence folders and the public template links.
- 20-40m: write a daily learning note with two known facts and two open questions.
- 40-60m: update progress only for checks actually completed; generate and review the dashboard.

E - Review and buffer (1h)

- Use remaining time for the earliest blocked prerequisite; do not add features.
- Record hours, observed learning, unresolved question and next action in the daily journal.
- Review evidence for sensitive data, update progress and publish a small truthful commit.

Day 2 task checklist: Environment, capabilities and pinned versions

Budget: 7 task hours + 1 hour buffer. All checkboxes start unchecked. Complete them only when the work is actually performed.

A - Inspect the Linux environment (2h)

- 0-30m: collect OS, kernel, architecture and resources using read-only commands in the PDF.
- 30-60m: record interfaces, routes, virtualization mode and console recovery.
- 60-90m: inspect required tool availability; classify missing tools versus missing kernel capabilities.
- 90-120m: fill D04; redact irrelevant host/user details before committing.

B - Validate the design (2h)

- 0-30m: inspect TUN availability and lab networking privileges; record actual outcomes.
- 30-60m: inspect SCTP support without assuming absence from loaded modules means unsupported.
- 60-90m: verify which addresses are reachable across host/guest/namespace boundaries.
- 90-120m: update D03 and D04, marking application connection checks pending until Day 4.

C - Choose versions and prerequisites (2h)

- 0-30m: read official setup guidance for your OS.
- 30-60m: choose explicit core and simulator commits/releases, then record full hashes.
- 60-90m: record compiler/build tools and required database/library versions.
- 90-120m: create D05 compatibility rows; say runtime compatibility is untested.

D - Preserve a build starting point (1h)

- 0-20m: record checkout, config, binary and log locations.
- 20-40m: save a VM snapshot or write a recovery procedure appropriate to the environment.
- 40-60m: fill D06 with persistence/reboot behavior; publish the reviewed manifest and progress.

E - Review and buffer (1h)

- Use remaining time for the earliest blocked prerequisite; do not add features.
- Record hours, observed learning, unresolved question and next action in the daily journal.
- Review evidence for sensitive data, update progress and publish a small truthful commit.

Day 3 task checklist: Build both components and reconcile configuration

Budget: 7 task hours + 1 hour buffer. All checkboxes start unchecked. Complete them only when the work is actually performed.

A - Core source build (2h)

- 0-30m: verify checkout hash and follow the OS-specific prerequisite instructions.
- 30-90m: build Open5GS; capture command, working directory, start/end and exit status privately.
- 90-120m: inspect first failures or record success in D07. Do not silently replace this build with a different version.

B - Core tests (2h)

- 0-30m: identify documented test entry points and prerequisites for this revision.
- 30-90m: run applicable tests and save full local output.
- 90-120m: publish a small result summary with command, pass/fail/skip counts and missing capabilities in D07. A skip is not a pass.

C - Simulator build (2h)

- 0-30m: confirm the pinned UERANSIM checkout and its build guidance.
- 30-90m: build and inspect the expected binaries/help output.
- 90-120m: fill D08 with actual outcomes and explicitly pending runtime verification.

D - Configuration consistency (1h)

- 0-20m: select example configs for this revision; record provenance.
- 20-40m: compare PLMN, TAC, DNN, slice selection, transport addresses and synthetic subscriber agreement in D09.
- 40-60m: publish the agreement checklist without keys; record services/startup prerequisites.

E - Review and buffer (1h)

- Use remaining time for the earliest blocked prerequisite; do not add features.
- Record hours, observed learning, unresolved question and next action in the daily journal.
- Review evidence for sensitive data, update progress and publish a small truthful commit.

Day 4 task checklist: Registration, session and actual user-plane traffic

Budget: 7 task hours + 1 hour buffer. All checkboxes start unchecked. Complete them only when the work is actually performed.

A - Core readiness (2h)

- 0-30m: create a run ID and note configuration hashes.
- 30-60m: start database and required core functions following the selected setup.
- 60-90m: verify actual listeners/readiness and record failures.
- 90-120m: check UPF data-endpoint routing and readiness; add precise evidence locations to D10.

B - gNB connection (2h)

- 0-30m: verify gNB AMF address and intended namespace.
- 30-60m: start the simulated gNB; collect setup logs/capture.
- 60-90m: distinguish transport failure from protocol/config rejection.
- 90-120m: record the observed setup outcome in D10; troubleshoot only the first failing layer.

C - UE registration and PDU session (2h)

- 0-30m: provision one synthetic subscriber privately and check UE agreement.
- 30-60m: start UE and record registration evidence.
- 60-90m: record session request/outcome separately from registration.
- 90-120m: record assigned UE address/interface and visible session identifiers in D11; mark unavailable fields explicitly.

D - Traffic proof (1h)

- 0-15m: select a controlled IP endpoint and confirm the probe source uses the UE interface.
- 15-40m: send traffic and observe it at the UE and user-plane path, including replies.
- 40-60m: publish a sanitized command/result/capture reference in D11; explain why host-network bypass was excluded.

E - Review and buffer (1h)

- Use remaining time for the earliest blocked prerequisite; do not add features.
- Record hours, observed learning, unresolved question and next action in the daily journal.
- Review evidence for sensitive data, update progress and publish a small truthful commit.

Day 5 task checklist: Golden run, source walk, repeat and public demonstration

Budget: 7 task hours + 1 hour buffer. All checkboxes start unchecked. Complete them only when the work is actually performed.

A - Golden captured run (2h)

- 0-30m: start from the documented stopped state, assign run ID and start relevant captures.
- 30-75m: perform startup, UE/session establishment and the same controlled traffic.
- 75-105m: stop captures cleanly and check interfaces, timestamps and useful packet content.
- 105-120m: index artifacts locally and prepare reviewed public excerpts.

B - Timeline and source explanation (2h)

- 0-75m: fill all eight D12 timeline rows using observed/inferred/unavailable labels.
- 75-105m: locate one relevant source handler/state transition at a pinned commit.
- 105-120m: fill D13, distinguishing source interpretation from runtime proof.

C - Controlled repeat (2h)

- 0-30m: stop cleanly and record starting state; use a new run ID.
- 30-75m: repeat the same procedure and traffic probe.
- 75-105m: compare outcomes and explain permitted identifier/timestamp differences; preserve any failure.
- 105-120m: complete D14 with the reproduction sequence and exact repeat outcome.

D - Demonstration and review (1h)

- 0-15m: prepare five-minute topology-to-evidence demonstration.
- 15-30m: present to a peer if available or record locally; label recorded output honestly.
- 30-45m: complete D15 learning summary and final outcome.
- 45-60m: update progress, inspect the showcase and publish the supported results.

E - Review and buffer (1h)

- Use remaining time for the earliest blocked prerequisite; do not add features.
- Record hours, observed learning, unresolved question and next action in the daily journal.
- Review evidence for sensitive data, update progress and publish a small truthful commit.

Fifteen deliverables people can inspect

Days 1-2 / decisions and environment

D01 - Scope: one-UE goal, five backlog items, exclusions and success criteria. Show a concise Markdown note; mark it done only when the decisions are filled in.

D02 - Starting point: four demonstrated skill checks with evidence, gaps and reuse choices. Show what you know and what you still need to verify.

D03 - Topology: actual Linux environment, logical links, addresses/namespaces and intended return path. Replace the template sketch with your real setup.

D04 - Environment: OS/kernel/resources and actual capability checks. **D05 - Versions:** exact revisions/tools and untested compatibility. **D06 - Recovery:** starting-state snapshot or reproducible restoration notes.

Days 3-4 / builds and observed behavior

D07 - Core build/tests: exact commands, outcomes and failed/skipped tests. **D08 - Simulator build:** build/binary checks and pending runtime checks. **D09 - Config agreement:** compare fields and synthetic provisioning without exposing credentials.

D10 - Registration: one run with separate core readiness, gNB setup and UE registration evidence. **D11 - Session/traffic:** assigned address plus packets and replies using the actual UE path, with host bypass excluded.

Day 5 / understanding and reproduction

D12 - Timeline: eight rows with timestamp, pointer, fact and uncertainty. **D13 - Source walk:** one handler at a pinned commit and an explanation of runtime proof versus source interpretation.

D14 - Repeat: starting state, commands, two run IDs and differences. **D15 - Review:** a five-minute demo, three lessons, one corrected assumption and the honest final outcome.

Where to put them

Every item has a dedicated template under weeks/week-01/deliverables in GitHub. Link reviewed screenshots or summaries from that template. Raw logs/captures remain private by default. The public showcase links completed evidence; unfinished entries explicitly say evidence pending.

Planning files are already supplied. All 15 technical deliverables begin as not started. Filling a real scope note can complete D01; uploading an empty build template cannot complete D07.

Make progress visible in GitHub

One source of truth

The repository's progress.json stores each deliverable's status. PROGRESS.md and the Week 1 showcase are generated from it. Five daily issues are editable checklists; issue closure does not automatically change the dashboard.

End-of-day update, step by step

1. Perform the task. Save raw evidence privately with a run ID and exact environment.
2. Fill the matching Markdown template with observed results, reproduction steps and limits. Remove the unfinished-template notice only when the record is actually filled.
3. Review publishable excerpts/screenshots. Remove secrets, real subscriber data and unrelated personal/employer material. A screenshot is optional; evidence-linked text is often clearer.
4. Edit that deliverable in progress.json: status, update date and notes. If blocked, state the reason and next diagnostic check.
5. To mark done, also set evidence_ready to true and fill evidence links, result and limitations. Link actual work, not a planned demonstration.
6. Run `python scripts/update_progress.py`, then `python scripts/update_progress.py --check` from the repository root. Use the appropriate Python command for your system.
7. Inspect git status and the diff. Stage named files, inspect the staged diff, commit a truthful description and push.
8. Add the evidence/commit link to the day's issue and check only the steps performed. Close it only when the day's exit check passes.

What the percentage means

The dashboard counts completed deliverables out of 15. It is not a measure of professional mastery or elapsed effort. A blocked task can still produce useful learning notes while remaining incomplete.

What GitHub checks mean

The workflow validates status data, file paths, completion metadata and generated-page consistency. It does not run Open5GS, inspect packet correctness or independently verify your claims. A green check means the learning record is consistent, not that the lab works.

Show learning through a small case study

Publish one useful daily update

Use this structure: question; setup; observation; interpretation; result; limitation; next check. Link the corresponding deliverable and commit. Prefer one clear technical explanation over a long list of tools used.

Example structure - not an actual result

"Today I tested whether [specific procedure] works in [pinned environment]. I used [command/trace]. The evidence shows [observed fact], so I concluded [bounded interpretation]. I still cannot establish [limitation]. Next I will test [one hypothesis]. Evidence: [deliverable link]."

Replace every bracket with your own work. Do not publish it as an achieved outcome before performing the task.

What to show each day

Day 1: a scope decision and annotated topology. **Day 2:** a concise environment/version manifest plus capability gaps. **Day 3:** build/test summaries and one error explanation if relevant. **Day 4:** a registration/session/data evidence chain. **Day 5:** a five-minute demonstration, source explanation and repeat result.

Make the Friday showcase understandable

Start with what currently works. Link the strongest evidence in D10-D14, state whether the repeat passed and show the limitations. Add a short "what I learned" section in D15. A visitor should be able to distinguish demonstrated behavior, a hypothesis and future work in under two minutes.

Review before every push

Check each staged file for tokens, authentication keys, employer material, real subscriber identifiers, private network details and unreviewed logs. The repository ignores common raw-evidence paths but cannot guarantee that a hand-copied secret is safe. Publish synthetic or deliberately sanitized evidence only.

Sharing the link

Share the repository or a specific completed deliverable when ready. This task publishes the planning repository and templates; it does not post claims on LinkedIn or message other people. Future posts should describe the results actually achieved.

Your first action: open the Day 1 guide and fill D01 with the precise scope. That is the first learning deliverable to complete and show publicly.